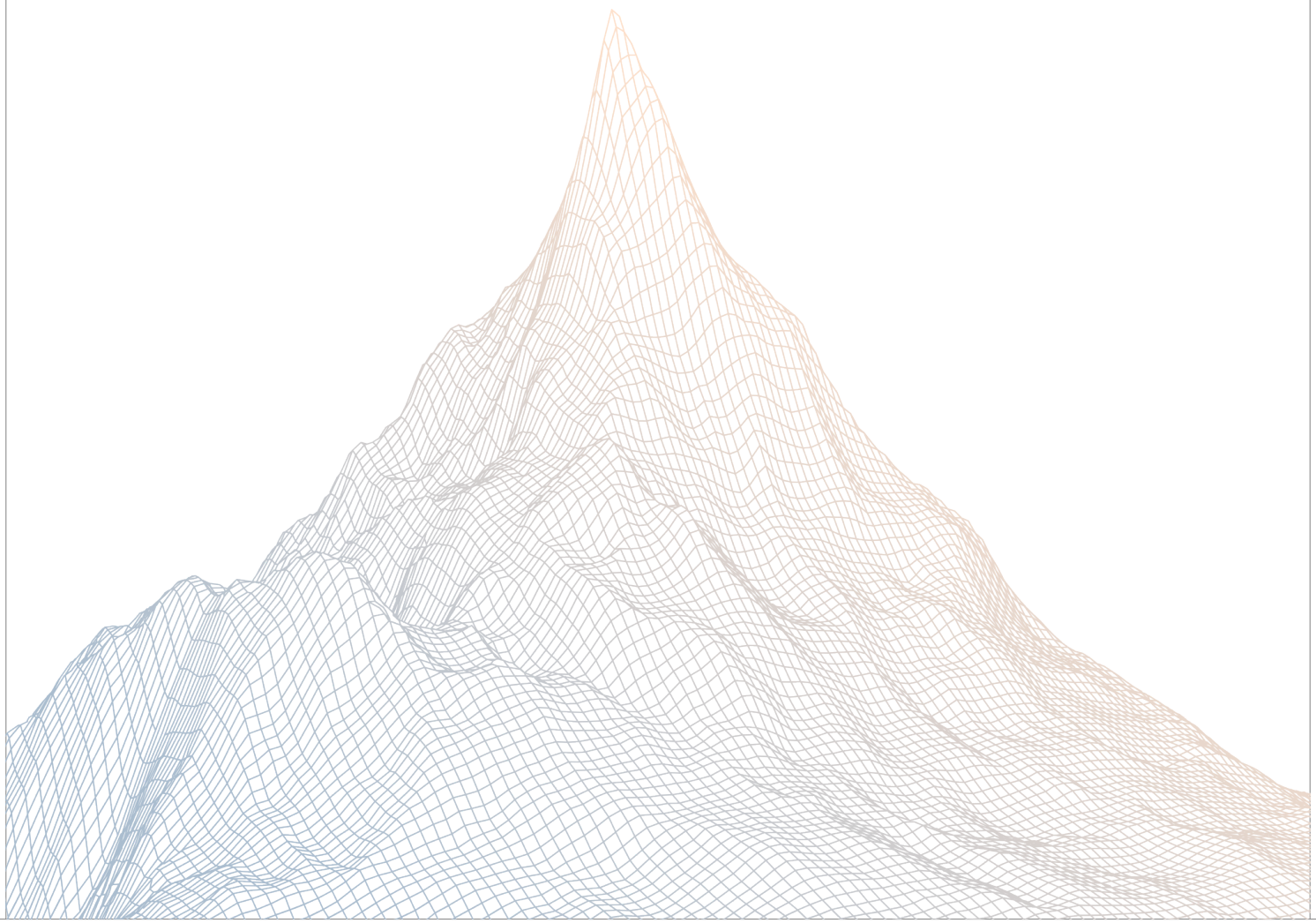
Zenith

Ammplify

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES: April 7th to April 9th, 2026

AUDITED BY: ast3ros
ether_sky

Contents

1 Introduction 2

1.1 About Zenith 3

1.2 Disclaimer 3

1.3 Risk Classification 3

2 Executive Summary 3

2.1 About Ammplify 4

2.2 Scope 4

2.3 Audit Timeline 5

2.4 Issues Found 5

3 Findings Summary 5

4 Findings 6

4.1 Medium Risk 7

4.2 Low Risk 10

4.3 Informational 20

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at https://zenith.security.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Ammplify

Ammplify is an on-chain derivative-structuring service for dapps. We innovate with novel uses of traditional AMM liquidity to create and underwrite retail-oriented derivatives for partners like the prediction markets for memecoin launchpads, non-liquidating money markets, alt-yielding stablecoin vaults, and liquidation-free leveraged farming. Ammplify aims to be at the nexus of risk-on yields in DeFi, providing higher yields via diverse dapp integrations and the most sophisticated LP analysis anywhere.

2.2 Scope

The engagement involved a review of the following targets:

Target	ItosOptions
Repository	https://github.com/itos-finance/ItosOptions
Commit Hash	`dc4d618a1e99099a2cb2ef9ec4eb52a3568baf24`
Files	`src/*/.sol`

Target	Mitigation Review
Repository	https://github.com/itos-finance/ItosOptions
Commit Hash	`916907158e8fec12c3a8f649dec53bffcf107382`
Files	`src/*/.sol`

2.3 Audit Timeline

April 7, 2026	Audit start
April 9, 2026	Audit end
April 18, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	1
Low Risk	7
Informational	7
Total Issues	**15**

3

Findings Summary

ID	Description	Status
M-1	Path-dependent settlement in sell versus buy yields different netting outcomes for identical trades	Resolved
L-1	Expiry can be extended indefinitely, locking seller collateral beyond agreed terms	Resolved
L-2	OLongToken.totalSupply overstates supply after exercise	Resolved
L-3	OToken hardcodes 18 decimals regardless of risk token	Resolved
L-4	Bulletin size limited to int128 while OPair accepts uint128	Acknowledged
L-5	The fee calculation should always be rounded up	Resolved
L-6	Updating the expiry should not be allowed after the current expiry has passed	Resolved
L-7	There is a risk of unfair claiming	Resolved
I-1	Some functions are missing the nonReentrant modifier	Resolved
I-2	Constructor does not enforce minimum exercise window	Resolved
I-3	The fee on transfer tokens are not supported	Resolved
I-4	There is an unused error	Resolved
I-5	Quotes should specify their own minimum allowed amounts	Acknowledged
I-6	Claiming an order drains the preferred payout pool	Acknowledged
I-7	Unused variable and import	Resolved

4

Findings

4.1 Medium Risk

A total of 1 medium risk findings were identified.

[M-1] Path-dependent settlement in sell versus buy yields different netting outcomes for identical trades

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- OPair.sol#L260-L326

Description:

`_settleNetted` splits a netted short into unexercised and exercised portions based on the current `totalSold` and `totalExercised`, preferring unexercised first:

```
    function _settleNetted(address account, uint256 toNet) internal {
>>>     uint256 totalUnexercised = totalSold - totalExercised;
        uint256 fromUnexercised = toNet > totalUnexercised
            ? totalUnexercised
            : toNet;
        uint256 fromExercised = toNet - fromUnexercised;

        ...
    }
```

In `sell`, `_addShort` (and `totalSold += physicalSize`) executes before `_addLong`, so when the buyer's existing short is netted via `_settleNetted`, `totalSold` already includes the new seller's freshly deposited collateral.

```
    function sell(
        address funderAddr,
        address buyerSigner,
        uint128 size,
        uint128 fill,
```

```
        uint128 premiumPerUnit,
        uint256 validTillTimestamp,
        bool allowPartialFill,
        bytes calldata signature
    ) external beforeDepositDeadline nonReentrant {
        ...

>>>     uint256 physicalSize = _addShort(seller, fill); // _addShort is
    called first
        if (physicalSize > 0) {
            depositToken.safeTransferFrom(
                seller,
                address(this),
                _depositAmount(physicalSize)
            );
>>>         totalSold += physicalSize;
        }

>>>     uint256 buyerNetted = _addLong(buyerSigner, fill); // _addLong is
    called after, so _settleNetted used new collateral

        ...
    }
```

This inflates the unexercised pool visible to the netting logic:

```
    function _settleNetted(address account, uint256 toNet) internal {
>>>     uint256 totalUnexercised = totalSold - totalExercised; // inflate by
    new seller collateral
        uint256 fromUnexercised = toNet > totalUnexercised
            ? totalUnexercised
            : toNet;
        uint256 fromExercised = toNet - fromUnexercised;

        ...
    }
```

In `buy`, the order is reversed. `_addLong` runs before `_addShort`, so netting sees the pre-trade pool state:

```
    function buy(
        address sellerFunderAddr,
        address sellerSigner,
        uint128 size,
        uint128 fill,
```

```
        uint128 premiumPerUnit,
        uint256 validTillTimestamp,
        bool allowPartialFill,
        bytes calldata signature
    ) external beforeDepositDeadline nonReentrant {
        ...

>>>     uint256 buyerNetted = _addLong(buyer, fill); // _addLong is called
    first, so _settleNetted used pre-trade pool state

>>>     uint256 physicalSize = _addShort(sellerSigner, fill); // _addShort
    is called after
        ...
    }
```

It means that for an identical trade with the same parties and sizes, the deposit-token vs swap-token split credited to `settledDepositToken` / `settledSwapToken` differs depending on which entry point is used.

When the unexercised pool is small relative to the fill, routing through `sell` can shift settlement entirely into deposit tokens (unexercised), while routing through `buy` would settle in swap tokens (exercised). A buyer holding a legacy short can exploit this asymmetry by coordinating the entry path to receive whichever token is more valuable.

Recommendations:

Reorder `sell` to call `_addLong(buyerSigner, fill)` before `_addShort(seller, fill)`, matching the sequencing in `buy` so netting always uses the pre-trade pool state.

Ammplify: Resolved with `@6124ee3021 ...`

Zenith: Verified.

4.2 Low Risk

A total of 7 low risk findings were identified.

[L-1] Expiry can be extended indefinitely, locking seller collateral beyond agreed terms

SEVERITY: Low	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- OPair.sol#L523-L527

Description:

`extendExpiry` only requires `newExpiry > expiry`. The factory owner can unilaterally extend collateral lockup far beyond the maturity sellers agreed to when they entered positions, worsening their terms without consent. Sellers price options based on expected lockup duration. An arbitrary extension increases their opportunity cost and market exposure without compensation. Since `claim` requires `afterExpiry`, sellers cannot exit until the admin-chosen date.

```
    function extendExpiry(uint256 newExpiry) external onlyFactoryOwner {
        if (newExpiry <= expiry) revert NewExpiryNotLater();
>>>     expiry = newExpiry;
        emit ExpiryExtended(newExpiry);
    }
```

Recommendations:

Only allow extensions when `totalSold == 0` so no existing positions are affected.

Ammplify: Resolved with `@f40b4bae6a ...`

Zenith: Verified.

[L-2] OLongToken.totalSupply overstates supply after exercise

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target

- OPair.sol#L44

Description:

`OToken.totalSupply` returns `pair.totalSold`, which tracks active short positions. When a buyer exercises, `netPosition[msg.sender]` decreases (reducing actual long exposure) and `totalExercised` increments, but `totalSold` remains unchanged until sellers claim.

As a result, `OLongToken.totalSupply` reports a value that includes exercised contracts whose corresponding longs no longer exist, overstating the circulating long supply.

```
function totalSupply() public view returns (uint256) {
    return pair.totalSold();
}
```

Recommendations:

Return `totalSold - totalExercised` for OLongToken.

Ammplify: Resolved with `@ccce577de8 ...`

Zenith: Verified.

[L-3] OToken hardcodes 18 decimals regardless of risk token

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target

- OPair.sol#L31

Description:

`OLongToken` and `OShortToken` hardcode `decimals = 18`, but their `balanceOf` returns values derived from `netPosition`, which is denominated in risk-token units. When the risk token has fewer than 18 decimals (e.g. WBTC with 8), wallets and frontends that rely on decimals will misinterpret the magnitude of a user's position by a factor of `10^(18 - riskDecimals)`.

```
abstract contract OToken is IERC20 {
    IOPair public immutable pair;

    string public name;
    string public symbol;
>>> uint8 public constant decimals = 18;
```

Recommendations:

Read the risk token's decimals at construction and store it, rather than hardcoding 18

Ammplify: Resolved with `@c42c8c6b16 ...`

Zenith: Verified.

[L-4] Bulletin size limited to int128 while OPair accepts uint128

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- Bulletin.sol#L38

Description:

`Bulletin.post` takes size as `int128` (max positive value 2^127 - 1).

```
    function post(
        address vault,
        address signer,
        address funder,
        uint128 premiumPerUnit,
>>>     int128 size,
        uint256 validTillTimestamp,
        uint256 nonce,
        bytes calldata signature
    ) external {
        ...
    }
```

But `OPair.sell` and `OPair.buy` accept size as `uint128` (max 2^128 - 1). This means orders with a size in the range `[2^127, 2^128 - 1]` cannot be posted to the Bulletin, even though they are valid in OPair.

```
    function sell(
        address funderAddr,
        address buyerSigner,
>>>     uint128 size,
        uint128 fill,
        uint128 premiumPerUnit,
        uint256 validTillTimestamp,
        bool allowPartialFill,
        bytes calldata signature
    ) external beforeDepositDeadline nonReentrant {
```

```
    function buy(
        address sellerFunderAddr,
        address sellerSigner,
>>>     uint128 size,
        uint128 fill,
        uint128 premiumPerUnit,
        uint256 validTillTimestamp,
        bool allowPartialFill,
        bytes calldata signature
    ) external beforeDepositDeadline nonReentrant {
```

Recommendations:

Accept `size` as `uint128` and add a separate bool `isBid` parameter to encode direction.

Ammplify: Acknowledged. I've always thought of the transactor as beholden to the bulletin, not the other way around. In the end, I think this is such a minor difference with no security risk, so I'll just leave it.

[L-5] The fee calculation should always be rounded up

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- OPair.sol

Description:

Now, the fee calculation is rounded down.

```
uint256 totalPremium = (uint256(premiumPerUnit) * uint256(fill)) / 1e18;
uint256 fee = (totalPremium * FEE_BPS) / BPS;
```

Recommendations:

The `premium` and `fee` calculations should be rounded up.

Ammplify: Resolved with `@4de06aef28 ...`

Zenith: Verified.

[L-6] Updating the expiry should not be allowed after the current expiry has passed

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- OPair.sol

Description:

A new future `expiry` can still be set after the current `expiry` has passed, which may lead to several unexpected issues.

```
function extendExpiry(uint256 newExpiry) external onlyFactoryOwner {
    if (newExpiry <= expiry) revert NewExpiryNotLater();
    expiry = newExpiry;
    emit ExpiryExtended(newExpiry);
}
```

Recommendations:

```
function extendExpiry(uint256 newExpiry) external onlyFactoryOwner {
function extendExpiry(uint256 newExpiry)
        external beforeExpiry onlyFactoryOwner {
    if (newExpiry <= expiry) revert NewExpiryNotLater();
    expiry = newExpiry;
    emit ExpiryExtended(newExpiry);
}
```

Ammplify: Resolved with `@3717038bd0 ...`

Zenith: Verified.

[L-7] There is a risk of unfair claiming

SEVERITY: Low	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Medium

Target

- OPair.sol

Description:

There is currently no restriction between `depositDeadline` and `exerciseEarliest`, which allows new `deposits` to be made after some previous `orders` have been exercised. In this situation, the `buy` function introduces a risk of unfair claiming. For example, suppose:

- `exerciseEarliest` = 10
- `depositDeadline` = 20
- `expiry` = 30

`Sellers` deposit a total of `100 risk tokens`, and all `orders` are exercised after `exerciseEarliest`. Then, `buyers` should receive `risk tokens` and `sellers` should receive the corresponding `cash tokens` only after `expiry`.

However, if the `cash token` becomes more valuable than `risk tokens` before `depositDeadline`, a malicious user could call the `buy` function. This would immediately assign `cash tokens` (since all `orders` were exercised) and add `risk tokens` to the pool.

```
function _settleNetted(address account, uint256 toNet) internal {
    uint256 totalUnexercised = totalSold - totalExercised;  // 0
    uint256 fromUnexercised = toNet > totalUnexercised
        ? totalUnexercised
        : toNet;
    uint256 fromExercised = toNet - fromUnexercised;

    // Longs clear previously sold contracts.
    totalExercised -= fromExercised;
    totalSold -= toNet;

    settledDepositToken[account] += fromUnexercised;
    settledSwapToken[account] += fromExercised;
```

```
    emit Netted(account, fromUnexercised, fromExercised);
}
```

As a result, the `claiming pool` now contains both `risk tokens` and `cash tokens`.

Consequently, other `sellers` would receive both `risk` and `cash tokens` after `expiry`, even though they were only supposed to receive `cash tokens`. This creates an unfair advantage for the malicious users, as they can benefit from receiving `cash tokens` before `expiry` and could claim both `cash` and `risk tokens` after `expiry`.

Recommendations:

The first solution is to enforce that `exerciseEarliest` must be later than `depositDeadline`. If allowing `deposits` after `exercise` is an intended design, the following recommendation can address this issue.

```
function buy(
    address sellerFunderAddr,
    address sellerSigner,
    uint128 size,
    uint128 fill,
    uint128 premiumPerUnit,
    uint256 validTillTimestamp,
    bool allowPartialFill,
    bytes calldata signature
) external beforeDepositDeadline nonReentrant {
    ...
    // Buyer acquires a long position for `fill` units.
    uint256 buyerNetted = _addLong(buyer, fill);

    // Seller acquires a short position for `fill` units.
    uint256 physicalSize = _addShort(sellerSigner, fill);

    uint256 acquireAmt = _depositAmount(physicalSize); // 0 when fully netted

    // Seller acquires a short position for `fill` units.
    uint256 physicalSize = _addShort(sellerSigner, fill);

    uint256 acquireAmt = _depositAmount(physicalSize); // 0 when fully netted

    // Buyer acquires a long position for `fill` units.
    uint256 buyerNetted = _addLong(buyer, fill);
```

If a `buyer` already holds a `short position`, it indicates that they expect the `deposit token` to increase in value and want to recover their previously `deposited tokens`. So the deposited tokens by new `seller` should be added to the `totalSold` pool, and the buyer should be able to claim these tokens. This behavior is theoretically correct.

Ammplify: Resolved with `@6124ee3021 ...` and `@48db929fed ...`

Zenith: Verified.

4.3 Informational

A total of 7 informational findings were identified.

[I-1] Some functions are missing the nonReentrant modifier

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- OPair.sol

Description:

Some functions like `claim` and `claimFees` are missing the `nonReentrant` modifier.

Recommendations:

Add `nonReentrant` modifier to these functions.

Ammplify: Resolved with `@d636d93786 ...`

Zenith: Verified.

[I-2] Constructor does not enforce minimum exercise window

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- OPair.sol#L220

Description:

The constructor sets `exerciseEarliest = block.timestamp + 4 hours` without checking that this leaves at least a 3-hour exercise window before expiry. The same check `setExerciseEarliest` has via `ExerciseWindowTooNarrow`.

```
    constructor(
        address _riskToken,
        address _cashToken,
        uint128 _strike,
        uint256 _expiry,
        bool _isCall,
        uint128 _minDepositSize,
        string memory identifier,
        string memory symbol
    ) {
        ...
        // It'd be strange to create a contract that had a small deposit
    window.
        if (block.timestamp + 1 hours >= depositDeadline)
            revert ExpiryTooSoon();
>>>     exerciseEarliest = block.timestamp + DEFAULT_EXERCISE_BUFFER;
        DOMAIN_SEPARATOR = _domainSeparatorFor(address(this));
        ...
```

```
    function setExerciseEarliest(
        uint256 newEarliest
    ) external onlyFactoryOwner {
>>>     if (newEarliest > expiry - 3 hours) revert ExerciseWindowTooNarrow();
        exerciseEarliest = newEarliest;
        emit ExerciseEarliestUpdated(newEarliest);
```

```
    }
```

A pair created with an expiry ~4—7 hours out deploys successfully but may give buyers a near-zero exercise window.

Recommendations:

Enforce the same exercise-window bound in the constructor.

Ammplify: Resolved with `@7b77aa4819 ...`

Zenith: Verified.

[I-3] The fee on transfer tokens are not supported

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- OPair.sol

Description:

Now, the `fee-on-transfer` tokens are not supported as it breaks the accounting.

Recommendations:

The documentation should clearly specify that `fee-on-transfer` tokens are not supported.

Ammplify: Resolved with is `@1923b529a7 ...`

Zenith: Verified.

[I-4] There is an unused error

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- OPair.sol

Description:

Below error is not used.

```
abstract contract OToken is IERC20 {
    error OnlyPair();
```

Recommendations:

```
abstract contract OToken is IERC20 {
    error OnlyPair();
```

Ammplify: Resolved with @f9255f10a4 ...

Zenith: Verified.

[I-5] Quotes should specify their own minimum allowed amounts

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- OPair.sol

Description:

Currently, all `quotes` use the global `minimum deposit amount` for validation.

```
function sell(
    address funderAddr,
    address buyerSigner,
    uint128 size,
    uint128 fill,
    uint128 premiumPerUnit,
    uint256 validTillTimestamp,
    bool allowPartialFill,
    bytes calldata signature
) external beforeDepositDeadline nonReentrant {
    if (validTillTimestamp < block.timestamp) revert QuoteExpired();
    if (fill == 0) revert ZeroSize();
    if (fill > size) revert FillExceedsQuotedSize();
    if (!allowPartialFill && fill < size) revert PartialFillNotAllowed();
    if (fill < minDepositSize) revert BelowMinDeposit();
```

However, this may be too low for certain `quotes`. For example, a `quote owner` might want `orders` to be filled between `10,000` and `20,000`, but the global `minimum deposit` could be as low as `10`, which is insufficient.

Recommendations:

It is common to specify a minimum allowed amount for each `quote` individually.

Ammplify: Acknowledged. Bulletin quotes don't allow partial fills. RFQ quotes are size insensitive on the low end up to a point. As long as there is some economic value greater

than the cost to exercise it's worth buying.

[I-6] Claiming an order drains the preferred payout pool

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- OPair.sol

Description:

Some `orders` have been `excised` while others have not, making it difficult to accurately match `long` and `short` positions. After an expiry, `short` holders are allowed to claim their tokens through the `claim` function.

```
function claim(
    address recipient,
    uint128 size,
    bool preferExercised
) external afterExpiry {
    if (preferExercised) {
        fromExercised = size > totalExercised ? totalExercised : size;
        fromUnexercised = size - fromExercised;
    } else {
        fromUnexercised = size > totalUnexercised ? totalUnexercised : size;
        fromExercised = size - fromUnexercised;
    }
}
```

This function lets users withdraw from either the `risk token pool` or the `cash token pool` based on their preference. However, this creates an unfair scenario for later claimants, as early claimers may drain the more valuable pool first, leaving less favorable assets for others.

Recommendations:

Claimers should receive risk tokens and cash tokens on a pro rata basis according to `totalSold` and `totalExcised`.

Ammplify: Acknowledged. It's design choice.

[I-7] Unused variable and import

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- OPair.sol#L153
- OPairFactory.sol#L6-L8

Description:

`DOMAIN_SEPARATOR` in `OPair` is declared as a public immutable and set in the constructor, but is never read internally. `SigVerifier._domainSeparatorFor(address(this))` recomputes it on every call.

```
bytes32 public immutable DOMAIN_SEPARATOR;
```

Separately, `OPairFactory` imports `IERC20Metadata` but never references it.

```
import {
    IERC20Metadata
} from "@openzeppelin/contracts/token/ERC20/extensions/IERC20Metadata.sol";
```

Recommendations:

Remove the unused variable and import

Ammplify: Resolved with `@dd88f457c0 ...` and `@f9255f10a4 ...`

Zenith: Verified.